The Nurse Scheduling Problem

Francisco Fonseca, 102492*
Gonçalo Rua, 102604*
franciscodiasfonseca@tecnico.ulisboa.pt
goncalonrua@tecnico.ulisboa.pt
Instituto Superior Técnico
Lisbon, Portugal

1 Description of the Problem

The Nurse Scheduling Problem (NSP) involves creating an optimal shift schedule for nurses working in a hospital over a given period of days, subject to some constraints:

(1) Each shift requires a minimum number of nurses for the normal functioning of the hospital.
(2) Each shift must have at least one nurse with managerial qualifications.
(3) A nurse's assigned shifts on any given day must be continuous, with no gaps in between.
(4) Each nurse has a minimum and maximum daily workload, defined by the number of assigned shifts.
(5) The total number of shifts that can be assigned to a nurse for the entire period is limited.
(6) A nurse cannot be assigned to shifts when it's known they're unavailable to work.
(7) The schedule must consider nurses' shift preferences. An optimal solution aims to minimize the total cost associated with the assignments.

2 Description of the Encoding

We first define a set of variables to represent the schedule and then encode the problem's constraints as clauses.

2.1 Definition of Variables

- Let D be a positive integer representing the number of days in the scheduling period.
- Let S be a positive integer representing the number of shifts per day, all with the same duration, that cover the full 24 hours. For example, if $S = 4$, each shift will have 6 hours.
- Let $R_{s,d}$ be the minimum number of nurses required for shift s ($1 \leq s \leq S$) on day d ($1 \leq d \leq D$).
- Let $N = \{N_1, N_2, \ldots, N_n\}$ be the set of n available nurses.
- Let $M \subseteq N$ be the subset of nurses with managerial qualifications.
- Let $k_{\min}$ and $k_{\max}$ be the minimum and maximum number of shifts for nurse N_k ($1 \leq k \leq n$) in any given day.
- Let H_k be the maximum total number of shifts that can be assigned to nurse N_k over the entire period.
- Let $P_{k,s,d}$ be the cost of assigning nurse N_k to shift s on day d. If $P_{k,s,d} = -1$, nurse N_k is not available for that shift. All other costs are positive integers representing preferences (a lower cost is better).

2.2 Modeling Constraints

This problem can be modeled and solved using SMT.

We define a set of integer variables that describe the schedule more abstractly than using boolean assignments. For each nurse $k \in \{1, \ldots, n\}$ and each day $d \in \{0, \ldots, D-1\}$, we define two core integer variables:

- $\mathbf{start_{k,d}}$: The 0-indexed starting shift for nurse k on day d.
- $\mathbf{num_{k,d}}$: The total number of consecutive shifts nurse k works on day d.

To simplify the main constraints, we first define a helper predicate, $Works(k, d, s)$, which is true if nurse k is working on shift s of day d. The shift index s is 0-indexed, $s \in \{0, \ldots, S-1\}$.

$$Works(k, d, s) \equiv (start_{k,d} \leq s) \wedge (s < start_{k,d} + num_{k,d}) \quad (1)$$

The constraints are then defined as follows, using an indicator function $\mathbb{I}(\cdot)$ which is 1 if its argument is true and 0 otherwise. This corresponds to the *If(condition, 1, 0)* construct in the code.

(1) **Domain Constraints:** The variables must stay within physically possible bounds. For all $k \in \{1, \ldots, n\}$ and $d \in \{0, \ldots, D-1\}$:

$$0 \leq start_{k,d} < S \quad (2)$$

$$0 \leq num_{k,d} \leq S \quad (3)$$

$$start_{k,d} + num_{k,d} \leq S \quad (4)$$

*Both authors contributed equally to this work.

(2) **Minimum Nurses per Shift:**

$$\sum_{k=1}^{n} \mathbb{I}(Works(k,d,s)) \geq R_{s,d}, \quad \forall d \in \{0,\ldots,D-1\}, s \in \{0,\ldots,S-1\} \tag{5}$$

(3) **Manager Requirement:**

$$\sum_{k \in M} \mathbb{I}(Works(k,d,s)) \geq 1, \quad \forall d \in \{0,\ldots,D-1\}, s \in \{0,\ldots,S-1\} \tag{6}$$

(4) **Shift Continuity:** This constraint is satisfied implicitly by the variable encoding. By defining a nurse's daily work as a single block of $num_{k,d}$ shifts starting at $start_{k,d}$, gaps within a work period are impossible. Therefore, no separate constraint is needed to enforce this rule.

(5) **Daily Workload:**

$$(L_k^{min} \leq num_{k,d} \leq L_k^{max}), \quad \forall k \in \{1,\ldots,n\}, d \in \{0,\ldots,D-1\} \tag{7}$$

(6) **Total Workload:**

$$\sum_{d=0}^{D-1} num_{k,d} \leq H_k, \quad \forall k \in \{1,\ldots,n\} \tag{8}$$

(7) **Unavailability:**

$$\neg Works(k,d,s), \quad \forall k,d,s \text{ where } P_{k,s,d} = -1 \tag{9}$$

(8) **Objective Function:**

$$\text{minimize} \sum_{d=0}^{D-1} \sum_{s=0}^{S-1} \sum_{k=1, P_{k,s,d} \geq 0}^{n} \mathbb{I}(Works(k,d,s)) \cdot P_{k,s,d} \tag{10}$$

3 Alternative Encodings

3.1 Boolean Encoding

Let $x_{k,s,d}$ be a Boolean variable such that $x_{k,s,d} = 1$ if nurse N_k is assigned to shift s on day d, and $x_{k,s,d} = 0$ otherwise.

(1) **Minimum nurses per shift:**

$$\sum_{k=1}^{n} x_{k,s,d} \geq R_{s,d}, \quad \forall s \in [1,S], d \in [1,D] \tag{11}$$

(2) **Manager requirement:**

$$\sum_{N_k \in M} x_{k,s,d} \geq 1, \quad \forall s \in [1,S], d \in [1,D] \tag{12}$$

(3) **Shift continuity:**

$$(\neg x_{k,s,d} \vee x_{k,s+1,d} \vee \neg x_{k,v,d}), \quad \forall N_k \in N, d \in [1,D], s \in [1,S-1], v \in [s+2,S] \tag{13}$$

(4) **Daily workload:**

$$k_{\min} \leq \sum_{s=1}^{S} x_{k,s,d} \leq k_{\max}, \quad \forall N_k \in N, d \in [1,D] \tag{14}$$

(5) **Total workload:**

$$\sum_{d=1}^{D} \sum_{s=1}^{S} x_{k,s,d} \leq H_k, \quad \forall N_k \in N \tag{15}$$

(6) **Unavailability:**

$$(\neg x_{k,s,d}), \quad \forall k,s,d \text{ where } P_{k,s,d} = -1 \tag{16}$$

(7) **Objective Function:**

$$\text{minimize} \sum_{d=0}^{D-1} \sum_{s=0}^{S-1} \sum_{k=1, P_{k,s,d} \geq 0}^{n} x_{k,d,s} \cdot P_{k,s,d} \tag{17}$$

3.2 Hybrid Encoding

The hybrid encoding seeks to combine the strengths of both the pure boolean and pure integer models. It uses integer variables to elegantly handle structural constraints like shift continuity and workload totals, while also using boolean variables to allow for fast, simple checks on a per-shift basis.

This model defines three sets of variables for each nurse k, day d, and shift s:

- A boolean variable $x_{k,d,s}$, which is true if nurse k is assigned to shift s on day d.
- An integer variable $start_{k,d}$ for the starting shift of nurse k on day d.
- An integer variable $num_{k,d}$ for the number of consecutive shifts worked by nurse k on day d.

The core of this model is a "bridge" constraint that links the boolean and integer variables, ensuring they are consistent. For all k, d, s:

$$x_{k,d,s} \iff (start_{k,d} \leq s) \wedge (s < start_{k,d} + num_{k,d}) \tag{18}$$

With this bridge in place, we can formulate each constraint using the most efficient variable type for the task:

(1) **Per-Shift Constraints:** The requirements for **minimum nurses per shift**, **managers requirement**, and **unavailability** are best expressed using the simple boolean variables. These constraints are identical to the pure boolean encoding.

(2) **Structural Constraints:** The rules for **daily workload** and **total workload** are handled by the more expressive integer variables. These constraints are identical to the first encoding. For **shift continuity**, it is satisfied by the bridge constraint linking $x_{k,d,s}$ to the single, continuous block defined by $start_{k,d}$ and $num_{k,d}$.

(3) **Objective Function:** The cost function is also defined using the simple boolean variables, summing the preferences for all assigned shifts.

$$\text{minimize} \sum_{d=0}^{D-1} \sum_{s=0}^{S-1} \sum_{k=1, P_{k,s,d} \geq 0}^{n} x_{k,d,s} \cdot P_{k,s,d} \tag{19}$$

4 Performance Comparison

TODO

5 Algorithm and Configuration

The encoding was implemented in Python, and the resulting formula was provided to Microsoft's Z3 SMT Solver [1], accessed via the `z3-solver` library. The `Optimize` class is used to handle the problem, as it is designed to find a model that not only satisfies all the hard constraints but also minimizes an objective function, which in this case is the total assignment cost. We set the solver strategy to use a Pareto approach (via `solver.set(priority='pareto')`) since we empirically observe that it provides a significant improvement in solving time compared to the default.

6 Installing and running

This project was implemented in Python. The sole dependency is z3-python, which can be installed via pip:

```
pip3 install z3-python
```

The program can be executed with the following command:

```
python3 project2.py < [input.txt] > [output.txt]
```

References

[1] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 337–340.